Save Point

Database Schema

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 23 October 2026

Single local SQLite database (`lib/storage/db.dart` holds the full DDL as a list of statements, applied once via `sqflite`'s `onCreate` callback). `PRAGMA foreign_keys = ON` is set on every connection, so all `ON DELETE CASCADE` relationships below are enforced.

All timestamp columns are `TEXT`, stored as UTC ISO-8601 strings (`DateTime.toUtc().toIso8601String()`). Repos always compute timestamps in Dart (never rely on a SQL-side default), so serialization is deterministic and easy to assert on in tests.

Tables

`folders`: the notebook's folder tree. `id, parent_folder_id (nullable, self-referencing, cascade), name, created_at`

`notes`: belongs to a folder. `id, folder_id -> folders.id (cascade), title, body, source_filename (nullable), sort_order (default 0), created_at, updated_at`

`sort_order` exists specifically so bulk-import's drag-to-reorder step means something after the fact: `NotesRepo.listByFolder()` orders by `sort_order` first, falling back to `title` for notes that share the default value (e.g. ones created individually rather than through bulk import).

`questions`: one row per generated (or manually edited) quiz question. `interval_index` (0 to 6, checked) is the single source of truth for scheduling; see `scheduling_algorithm.md`. `id, note_id -> notes.id (cascade), question_type ('short_answer' | 'multiple_choice'), prompt, expected_answer, interval_index (0-6), last_asked_at (nullable), next_due_at, created_at, active (0|1), is_manually_edited (0|1)`

Questions are soft-deletable in principle (`active` exists for exactly that), though nothing in the app currently sets it to 0. See `future_work.md`.

`question_choices`: multiple-choice options only; empty for short-answer questions. `id, question_id -> questions.id (cascade), choice_text, is_correct (0|1), sort_order`

`question_history`: append-only audit trail of every answer ever given. Never updated or deleted by the repo layer (`ReviewsRepo` has no update()/delete() method at all). This is the source of truth for future per-question/per-folder accuracy stats; `interval_index` alone can't reconstruct history since it's overwritten on every review. `id, question_id -> questions.id (cascade), asked_at, given_answer, is_correct (0|1), interval_index_before, interval_index_after, grading_method ('exact_match' | 'keyword_match' | 'mcq_exact')`

Design notes

- No ORM, plain `sqflite` + a thin repository class per table group (`lib/storage/*_repo.dart`). Each repo constructor takes a `Database` (dependency injection), which is what makes real in-memory SQLite fixtures in tests (via `sqflite_common_ffi`) fast and fully isolated.
- Indexes exist on every foreign key and on `questions.next_due_at` (the hot query path for `QuestionsRepo.listDue()`).
- No `question_source_notes` many-to-many table (the Python desktop app has one, for AI-generated questions that drew context from multiple linked notes). Not needed yet: this app doesn't have automatic note-linking ported over, so a question only ever has exactly one source note, via its plain `note_id` foreign key. See `future_work.md` .